

Flexibility and Microservices

Chapter 12 — Software Architecture (Pautasso)

- **What: flexibility** — how cheaply you can **change** structure, teams, and deployments as requirements move.
- **Why:** Lecture 11 separated **services** and **availability**; Lecture 12 asks how **fine-grained services** help or hurt evolution.

Definitions · **Flexibility** — ease of evolving the system (features, integrations, org) without excessive cost or risk. · **Microservices** — independently deployable services aligned with business capabilities, each owning its data and lifecycle.

Learning objectives

- **Modularity & monoliths** — *What*: cohesion/coupling; **modular monolith** as a stepping stone. *Why*: not every product needs **microservices** on day one.
- **Microservice traits** — *What*: **bounded contexts**, **database per service**, **API contracts**. *Why*: these choices trade **team autonomy** for **distributed-system** complexity.
- **Migration** — *What*: **strangler fig**, **branch by abstraction**. *Why*: big-bang rewrites rarely ship.
- **Pitfalls** — *What*: **distributed monolith**, chatty RPC, shared DB. *Why*: you can buy **all** the ops pain with **none** of the flexibility.

Definitions · **Modular monolith** — one deployable with **clear internal modules** and seams, without network between "services." · **Bounded context** — DDD boundary where a **ubiquitous language** and model stay consistent. · **Distributed monolith** — many services that must **deploy together** or share one database — tight coupling with network overhead.

Recap — Lectures 10–11

- **L10 — Scalability** — *What*: shards, caches, queues when load grows. *Why*: performance is not flexibility.
- **L11 — Availability & services** — *What*: SLIs/SLOs, breakers, replication, **vendor** coupling. *Why*: more services ⇒ more **partial failure** surfaces — flexibility must be **designed**, not accidental.

Definitions · **Partial failure** — some requests or nodes fail while others succeed (normal in distributed systems). · **Vendor coupling** — your evolution speed depends on another org's APIs, pricing, and roadmap.

Flexibility as a quality attribute

- **What: modifiability** in ISO-style terms — cost/time/risk to apply a **change** (feature, regulation, tech refresh).
- **Why:** investors and regulators care about **time-to-market** and **auditability** — architecture either **pays** or **taxes** each change.

Definitions · **Modifiability** — quality of how easily the architecture can be changed while preserving other qualities. · **Time-to-market** — calendar time from idea to production for a valuable increment.

What actually drives change?

- **What:** product experiments, **payment** rules, privacy law, new channels (voice, kiosk), **fraud** patterns, cloud price moves.
- **Why:** CityBite-style products churn **integrations** more than they churn **sorting algorithms** — boundaries should match **rates of change**.

Definitions · **Integration point** — boundary where two systems exchange data or events (API, queue, file). · **Rate of change** — how often a part of the domain or stack must change independently of the rest.

Coupling and cohesion (design vocabulary)

- **What: cohesion** — things that change together stay together; **coupling** — how much one module **forces** another to change or deploy.
- **Why:** microservices **do not remove** coupling — they **move** it to **network contracts** and **data consistency** stories.

Definitions · **Cohesion** — degree to which elements inside a module belong together (single reason to change). · **Coupling** — degree of interdependence between modules or services (API surface, shared schema, shared release).

Monolithic architecture

- **What:** **one** primary deployable; in-process calls; often **one** shared database schema.
- **Why:** simplest **transaction** and **refactor** story — grep-and-replace across a repo still works — great **early** velocity if discipline stays high.

Definitions · **Monolith** — application structured as a single deployable unit (even if internally layered). · **ACID transaction** — atomic, consistent, isolated, durable update spanning tables in one database (easy inside a monolith, hard across services).

Modular monolith (middle path)

- **What:** enforce **module** boundaries (packages, build targets, **no** “reach-around” imports) inside one binary.
- **Why:** practice **bounded contexts before** paying for **RPC** — if modules still fight, splitting to processes rarely **magics** autonomy.

Definitions · **Module boundary** — declared seam (package, library, compilation unit) with an explicit public API. · **Reach-around import** — bypassing a module’s public API to touch internals — hidden coupling.

When a monolith is still the right bet

- **What:** small team, unclear domain, need **strong consistency** everywhere, low operational maturity.
- **Why:** microservices multiply **repos, pipelines, logs, traces** — **flexibility** dies if nobody owns **on-call** for twelve services.

Definitions · **Operational maturity** — skills and automation for deploy, observe, secure, and restore distributed systems. · **Ownership** — a team responsible for a service's full lifecycle (code + pager + data).

Microservices — working definition

- **What:** **small** (not tiny-for-tiny's-sake) **autonomously deployable** services with **own data stores** and **APIs** — **smart endpoints, dumb pipes** intuition.
- **Why:** goal is **organizational** and **technical** parallelism — **Conway** cuts both ways.

Definitions · **Autonomous deployment** — change and release one service without coordinating a full-system redeploy. · **Smart endpoints, dumb pipes** — business logic lives in services; infrastructure messaging stays simple (e.g. HTTP, lightweight bus), not overloaded “magic” middleware.

Characteristics checklist (lecture view)

- **Own codebase & pipeline** — *What:* CI/CD per service. *Why:* **blast radius** of a bad deploy shrinks **if** contracts are stable.
- **Decentralized data** — *What:* **database per service**. *Why:* independent **schema evolution** — trades **joins** for **messages** or **API aggregation**.

Definitions · **CI/CD** — continuous integration and delivery: automated build, test, deploy pipelines. · **Schema evolution** — changing tables or documents over time without breaking all consumers at once.

Conway's Law (mirror)

- **What:** system design **reflects** org communication structure — and **reverse Conway** reshapes teams to get the architecture you need.
- **Why:** microservices without **team boundaries** often become **solo-maintained** sprawl — same person couples everything again.

Definitions · **Conway's Law** — organizations ship architectures that copy their communication lines. · **Reverse Conway maneuver** — intentionally structure teams to encourage desired architecture (e.g. two-pizza teams per service).

Bounded context and service boundaries

- **What:** align services to **business capabilities** — checkout, pricing, **dispatch**, fraud — not to **layers** (DAO service, “util” service).
- **Why:** wrong cuts $\Rightarrow$ **chatty** calls and **shared tables** — you recreated a distributed **procedure** call graph.

Definitions · **Business capability** — what the business does (e.g. “capture payment”), not a technical tier. · **Ubiquitous language** — shared precise terms inside a bounded context so code matches domain speech.

Database per service (and the cost)

- **What:** no **shared** row-level database between teams — integrate via **APIs** or **events**.
- **Why:** frees **schema** cadence — requires **sagas**, **outbox**, **eventual consistency** — product must accept **stale** reads sometimes.

Definitions · **Database per service** — each service owns its datastore; others access only via its API or published events. · **Eventual consistency** — replicas or services converge to the same state after a delay; reads may lag writes.

API contracts and evolution

- **What:** treat public APIs like **published products** — additive changes, **deprecation** windows, **consumer-driven** tests.
- **Why:** independent deploy is **fake** if every release **breaks** unknown mobile clients — **flexibility** needs **compat** rules.

Definitions · **Backward compatibility** — new servers still accept old clients without forced upgrades. · **Deprecation** — documented period where an old API remains supported before removal.

Distributed data & sagas (intro)

- **What:** cross-service business rules become **sagas** — **choreography** (events) or **orchestration** (central coordinator).
- **Why:** you **lost** two-phase commit across arbitrary SaaS — design **compensating** actions (cancel hold, refund, notify).

Definitions · **Saga** — sequence of local transactions with **compensating** steps if something fails later in the flow. · **Compensating transaction** — business action that semantically undoes or repairs a prior step (e.g. refund).

Network and partial failure (again)

- **What:** latency isn't constant; timeouts; **retries** need **idempotency**; **cascades** from Lecture 11 still apply.
- **Why:** “flexible” architecture that ignores **failure modes** becomes **fragile** — flexibility and **availability** share **design budget**.

Definitions · **Idempotency** — repeating an operation has the same effect as doing it once (safe retries). · **Timeout** — upper bound on how long a caller waits before treating a call as failed.

Flexibility vs other qualities

- **What:** finer services can improve **team scale-out** but hurt **latency, debuggability, consistency** — explicit **trade-off** table per journey.
- **Why:** teach PMs **cost curves** — ten deployables means ten **security** patches and ten **SLO** dashboards.

Definitions · **Trade-off** — improving one quality attribute often weakens another; architecture documents chosen balances. · **Observability** — logs, metrics, traces sufficient to infer internal state of distributed flows.

Strangler Fig pattern

- **What:** route **incrementally** from old monolith to new service behind a **facade** (proxy, API gateway, feature flag).
- **Why:** **dual-run** slices of traffic until confidence — **lowers** rewrite risk vs “stop the world” cutover.

Definitions · **Strangler Fig** — incrementally replace a legacy system by routing growing slices of functionality to new implementations. · **Facade** — stable external interface hiding internal decomposition or routing.

Branch by abstraction

- **What:** introduce an **interface** in the monolith; **two** implementations (old, new); **toggle** callers.
- **Why:** keeps trunk **integrating** while the new path hardens — pairs well with **strangler** routing.

Definitions · **Branch by abstraction** — use an abstraction boundary inside code to swap implementations without a long-lived source branch. · **Feature toggle** — configuration switch enabling or disabling a code path at runtime.

Dual-write / data migration (caution)

- **What:** for a time, **write** to old and new stores; **verify**; then **read** flip — needs **reconciliation** jobs.
- **Why:** flexibility projects **fail** in silent **drift** between stores — treat migration as **product** work, not only **DDL**.

Definitions · **Dual-write** — application writes the same logical change to two data stores during migration. · **Reconciliation** — batch or streaming job that detects and fixes differences between representations.

Anti-pattern — distributed monolith

- **What:** many processes but **one** release train, **shared** DB, or **synchronous ring** of calls for every request.
- **Why:** worst of both worlds — **network** faults **without** independent **team** or **schema** velocity.

Definitions · **Release train** — coordinated multi-service release where versions must move together (coupled lifecycle). · **Synchronous ring** — request path that must call many services in series before responding.

How small should a service be?

- **What:** “size” = **team ownership** + **clear context** + **deploy** cadence — **not** lines-of-code fetish.
- **Why:** **nano-services** explode **operational** cost — if two services **always** deploy together, merge them **until** the boundary is real.

Definitions · **Service granularity** — how much responsibility and data live in one deployable. · **Nano-service** — excessively fine split causing overhead disproportionate to autonomy gained.

Choreography vs orchestration (light)

- **What: choreography** — services react to **events** without a central brain; **orchestration** — **workflow engine** drives steps.
- **Why:** choreography scales **teams**; orchestration aids **visibility** for long **human-in-the-loop** flows — hybrid is common.

Definitions · **Choreography** — decentralized coordination where each service knows how to react to certain events. · **Orchestration** — centralized coordinator issues commands/steps to participants (workflow / saga manager).

DevOps and “you build it, you run it”

- **What:** same team ships **code** and carries **pager** — **SLO** per service from Lecture 11 vocabulary.
- **Why:** flexibility requires **fast feedback** — throwing code “over the wall” recreates **monolithic** coordination in **tickets**.

Definitions · **DevOps** — cultural and technical practices unifying development and operations for fast reliable delivery. · **Pager / on-call** — rotation answering production incidents for owned services.

Containers / Kubernetes (course tie-in)

- **What:** **Pod** + **Service** give a **deployable** unit and **stable** network name — fits **per-service** repos.
- **Why:** Lecture 9 primitives become the **runtime** for **many** small services — **config** and **secrets** multiply with count.

Definitions · **Pod** — smallest deployable unit in Kubernetes (one or more containers sharing network/storage context). · **Kubernetes Service** — stable virtual IP/DNS name load-balancing to healthy pods.

Example 1 — coupling (`example1_flexibility_coupling_citybite.py`)

- **What:** `OrderService` reaching into `payment` concrete class vs **injected port** (interface).
- **Why:** shows how **hidden imports** block **strangler** extraction — flexibility starts with **dependency direction**.

Definitions · **Port (hexagonal)** — interface your core defines for an external capability (payment, maps). · **Adapter** — implementation of that port for a specific vendor or in-memory test double.

Example 2 — API shape (`example2_flexibility_api_evolution_citybite.py`)

- **What:** **additive** JSON field vs **breaking** rename — client tolerance table in comments.
- **Why:** models **backward compatibility** rules you will put in **ASSIGNMENT** deliverables.

Definitions · **Additive change** — new optional field or behavior that old clients can ignore. · **Breaking change** — rename/remove/repurpose fields or semantics requiring coordinated client upgrades.

Assignment — CityBite “Split with care”

- **What: 150 pts** — contexts map, strangler plan, API evolution, Conway note, saga sketch, diagram.
- **Why:** practices **flexibility vocabulary** on the same product — see **ASSIGNMENT.md**.

Definitions · Deliverable — artifact you submit (markdown, diagram) proving analysis. · **Rubric** — grading criteria in **ASSIGNMENT.md** describing required depth.

References (chapter / course)

- Pautasso — **Chapter 12** PDF (`12_Flexibility_and_Microservices.pdf` in `../chapters/` or course bundle)
- **Building Microservices** (Newman), **Domain-Driven Design** (Evans / Vernon summaries), **Monolith to Microservices** (Richardson)

Definitions · **Domain-Driven Design (DDD)** — modeling approach aligning software structure with domain complexity and language. · **Quality attribute scenario** — stimulus–response measure for a quality (e.g. modifiability) used in architecture evaluation.

Takeaways

- **Flexibility is a trade** — **autonomy** and **independent schema** vs **operations** and **consistency** work.
- **Modular monolith** and **clean seams** reduce regret **before** process boundaries multiply.
- **Migrate with facades and abstractions** — **strangler** + **branch by abstraction** beat big-bang rewrites.

Definitions · **Big-bang rewrite** — replace entire system in one cutover — high risk, long feedback delay. · **Seam** — intentional boundary where one part can be replaced or extracted with controlled impact.

Questions?

- **What:** Chapter 12 PDF + course notes.
- **Why:** Next arc — deeper **DDD**, **integration styles**, or **hands-on** k8s labs depending on your syllabus.

Definitions · Syllabus — planned sequence of topics and activities for the course module. · **Integration style** — RPC, REST, messaging, or file-based patterns linking systems.